

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-a7c7cdaa289883eaf.jsonl`  
- SHA-256: `e99801a8e4242e7e0c9510e0008294613cf824ce0c0166a8a74646eef98ac314`  
- Source modified: `2026-06-21T14:01:11+00:00`  
- Imported at: `2026-07-05T16:48:23+00:00`  
- project: `wf_a2b97cff-dcd`  
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T13:56:24.636Z)

Adversarially VERIFY the load-bearing claims below about packaging/distributing our Python app. For EACH claim, try to REFUTE it (check feasibility, licensing accuracy, version facts, whether it actually applies to a torch+opencv+ffmpeg+FastAPI app). Default to "uncertain" if you cannot confirm. Use web search to check current facts.

TOPIC: prior-art-deepdive

CLAIMS:

```
[  
  {  
    "claim": "Our signature packaging problem is real and in-code: the WASB shuttle detector runs in a SEPARATE conda env via WSL ? `wasb_runner.py` literally builds commands `conda activate {self.cfg.conda_env}` with `conda_env` defaulting to 'wasb' (lines 36,74,162) ? while the main engine uses a different torch. An in-process escape hatch (`native_wasb_runner.py`) exists but is not production-verified.",  
    "confidence": "high",  
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py:36,74,162; base.py:46-54; native_wasb_runner.py"  
  },  
  {  
    "claim": "`uv` can express the exact torch CUDA/CPU `--index-url` constraint that our pyproject.toml header says PEP 621 cannot: define a PyTorch index in pyproject and route torch/torchvision to it via `[tool.uv.sources]` with `explicit=true`; backends follow https://download.pytorch.org/whl/{cpu,cu124,...}.",  
    "confidence": "high",  
    "source": "https://docs.astral.sh/uv/guides/integration/pytorch/ ; pyproject.toml:8-11,58-65"  
  },  
  {  
    "claim": "Pinokio (MIT-licensed) one-click 'sets up Python, Conda, virtual environments, GPU libraries and all required dependencies' in app-isolated envs and 'installs the latest version of FFMPEG', then launches a local web UI ? i.e. it natively handles multiple isolated torch/CUDA envs + ffmpeg, the shape of our problem.",  
    "confidence": "high",  
    "source": "https://github.com/pinokiocomputer/pinokio ; https://github.com/pinokiocomputer/pinokio/blob/main/LICENSE"  
  },  
  {  
    "claim": "ComfyUI's official desktop app is GPL-3.0, ships via NSIS (Win)/DMG (mac) Electron, bundles `uv` to install Python deps on first startup (not an embedded Python), uses pinned `.compiled` requirements, lets the user pick a models folder, and auto-updates the app + ComfyUI-Manager + uv on stable releases.",  
    "confidence": "high",  
    "source": "https://github.com/Comfy-Org/desktop"  
  },  
]
```

```

{
  "claim": "Ollama ships a ~5MB OllamaSetup.exe that installs without admin rights, auto-
starts a background service, auto-detects NVIDIA CUDA, pulls model weights on demand to
%USERPROFILE%\.ollama, runs offline after, and updates by running the new installer over the old
(models preserved).",
  "confidence": "high",
  "source": "https://docs.ollama.com/windows ; https://github.com/ollama/ollama/releases"
},
{
  "claim": "Fooocus and ComfyUI-portable ship a ~1GB+ ZIP with embedded Python and pre-
installed torch; extract + double-click run.bat downloads weights on first run and opens a
localhost web UI. Fooocus is GPL-3.0.",
  "confidence": "high",
  "source": "https://github.com/lillyasviel/Fooocus ;
https://github.com/lillyasviel/Fooocus/blob/main/LICENSE"
},
{
  "claim": "AUTOMATIC1111's 'one-click' is NOT batteries-included for non-tech users: it
requires pre-installing Python 3.10.6 + git, `git clone`, then webui-user.bat builds a venv and
pip-installs torch (~5-10 min first run); third-party launchers exist precisely because the
official one omits Python/git.",
  "confidence": "high",
  "source": "https://github.com/AUTOMATIC1111/stable-diffusion-webui ;
https://github.com/AUTOMATIC1111/stable-diffusion-webui/discussions/8020"
},
{
  "claim": "Frigate bundles its own ffmpeg inside the Docker image (at /usr/lib/ffmpeg, v7),
ships per-accelerator image variants (default OpenVINO/CPU, `-tensorrt` for NVIDIA), no longer
pre-bundles detection models (drop ONNX into /config/model_cache, converted on first boot), and
distributes via Docker tags (`stable`) on GHCR + a Home Assistant add-on; updates via image re-
pull or Watchtower.",
  "confidence": "high",
  "source": "https://docs.frigate.video/frigate/installation/ ;
https://docs.frigate.video/configuration/object_detectors/ ;
https://docs.frigate.video/frigate/updating/"
},
{
  "claim": "Frigate does not officially support Windows; it runs only via WSL2 + Docker
Desktop, and 'getting the GPU and/or Coral devices properly passed to Frigate may be difficult
or impossible' on Windows ? i.e. Docker+GPU on Windows is exactly what a non-tech user cannot
debug.",
  "confidence": "high",
  "source": "https://docs.frigate.video/frigate/installation/ ;
https://github.com/blakeblackshear/frigate/discussions/4375"
},
{
  "claim": "LM Studio (0.4+) decouples a headless daemon (`llmster`) from the GUI behind an
OpenAI-compatible HTTP server and versions its inference engine separately as swappable 'LM
Runtimes'; Jan is a Tauri app bundling llama.cpp serving 127.0.0.1:1337; GPT4All ships an OS
installer + in-app HuggingFace model hub + local server on :4891 ? all bundle the engine and
download weights on demand via a GUI.",
  "confidence": "high",
  "source": "https://lmstudio.ai/docs/app ; https://lmstudio.ai/blog/0.4.0 ;
https://github.com/janhq/jan ; https://docs.gpt4all.io/gpt4all_desktop/quickstart.html"
},
{
  "claim": "Owner decision D10 sets the delivery shape: a self-hosted webserver + UI the user
spins up (NOT a desktop app), pointing at local storage + GPU/CPU OR cloud locations; this
supersedes the prior DESKTOP_APP_PLAN.md.",
  "confidence": "high",
  "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39 (D10)"
},
{
  "claim": "There is currently NO Dockerfile in the engine, and the WASB torch-1.11 conda env
makes a containerized worker 'non-trivial' ? a noted gap blocking the Cloud Run+GPU serving

```

```

path.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:49,96,100"
}
]

```

RECOMMENDATION UNDER TEST: Adopt a layered, profile-matched strategy, not one installer. (1) FIRST, finish `native\_wasb\_runner.py` (the in-process torch path that retires the WSL/conda 'wasb' sidecar seen at wasb_runner.py:74,162). Almost every clean packaging story below collapses to ONE torch environment ? so removing the second incompatible env is the highest-leverage prerequisite, and it's already designed-for (base.py:46-54). (2) For the NON-TECH WINDOWS user, copy the Ollama+ComfyUI-desktop pattern: a small self-branded installer (license-clean Tauri shell, or even a plain NSIS/portable launcher ? NOT the GPL ComfyUI/Foocus code) that bundles `uv`, on first run uses `uv` with `[tool.uv.index]`/[tool.uv.sources]` to install torch/torchvision from the correct CUDA-or-CPU `--index-url` (the precise thing pyproject.toml says PEP 621 can't do), bundles an LGPL ffmpeg build (openh264/NVENC, no GPL x264 per D4), downloads the GCS WASB weights on first run with a progress bar (Ollama/GPT4All model-hub UX), then launches `python -m backend.main --server` and opens 127.0.0.1:8000 to the existing khelsutra React SPA. Update like Ollama: run a new installer over the old, weights preserved. (3) For the SERVER / cloud-serving profile, build the Frigate-style Docker image (bundled ffmpeg + per-accelerator variant) ? which ALSO closes the 'no Dockerfile' gap blocking Cloud Run+GPU. (4) As a near-zero-effort additional channel, publish a Pinokio install script (MIT, app-isolated envs + ffmpeg + GPU libs out of the box) so technically-curious users get one-click install without us owning a launcher. Do NOT make Docker-on-Windows the non-tech path: Frigate itself shows Docker+GPU passthrough on Windows is exactly what that user cannot debug.

2. assistant (2026-06-21T13:56:30.768Z)

I'll verify each claim adversarially. Let me start by examining the in-code claims (which I can verify directly), then web-check the external facts.

3. user (2026-06-21T13:56:31.743Z)

```

1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21     TrackNetRunner.
22     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24     wsl_tilde_quote
25
26     logger = logging.getLogger(__name__)
27
28     # Windows path to the inference wrapper we stage into WSL.
29     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
30
31     @dataclass
32     class WasbConfig:

```

```

32     """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33     distro: str = "Ubuntu"
34     repo_dir: str = "~/models/WASB-SBDT"
35     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36     conda_env: str = "wasb"
37     weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38     sport: str = "badminton"
39     wsl_stage_dir: str = "~/clips"
40     timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41     keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42     min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43     # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44     # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45     # extraction that dominates a long clip. Output is bit-identical to the PNG path
46     # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47     stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                   f"python -c \"import torch; print('torch', torch.__version__, "
76                   f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)

```

```

91         if reason:
92             logger.error("Cannot stage video into WSL: %s", reason)
93             return None
94         dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}"
96         {wsl_tilde_quote(dest)} && echo OK"
97         try:
98             res = self._wsl_bash(cmd)
99         except subprocess.TimeoutExpired:
100             logger.error("Staging video into WSL timed out.")
101             return None
102         if res.returncode != 0 or "OK" not in (res.stdout or ""):
103             logger.error("Failed to stage video into WSL: %s", (res.stderr or
104 res.stdout).strip())
105             return None
106         return dest
107
108 def _stage_infer_script(self) -> bool:
109     """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110     src_mnt = to_wsl_mnt_path(_INFER_WIN)
111     cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112 OK"
113     res = self._wsl_bash(cmd)
114     return res.returncode == 0 and "OK" in (res.stdout or "")
115
116 def run_predict(self, video_win_path: str, output_win_dir: str,
117                 video_id: Optional[str] = None) -> Optional[str]:
118     """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120     All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121     two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122     can never reuse each other's staged frames, resumable cache, or CSV.
123
124     """
125     # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126     # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127     # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
128     # the CSV landed inside WSL while Windows polled the repo-relative path.
129     output_win_dir = os.path.abspath(output_win_dir)
130     os.makedirs(output_win_dir, exist_ok=True)
131     # Normalize for cross-platform basename (tests use Windows paths on Linux).
132     video_win_path = str(video_win_path).replace("\\", "/")
133     base = os.path.basename(video_win_path)
134     stem, ext = os.path.splitext(base)
135     # Unique, content-derived key: same filename + different bytes -> different key.
136     key = f"{stem}__{video_id[:12]}" if video_id else stem
137     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
138     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
139
140     if not self._stage_infer_script():
141         logger.error("Could not stage wasb_infer.py into WSL.")
142         return None
143     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
144     if staged_video is None:
145         return None
146     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
147
148     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
149     # WSL stage filesystem is already low so the user can clean up before it fills.
150     # (Streaming never materializes the PNGs, so the guard only matters off the fast
151     path.)
152
153     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
154         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
155         if free is not None and free < self.cfg.min_free_gb:
156             logger.warning(
157                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame

```

```

extraction "
152             "may fill the disk ? consider running tools/cleanup_caches.py.",
153             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157         if self.cfg.stream_video:
158             frame_args = "--stream-video "
159         else:
160             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161         cmd = (
162             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163             f"python wasb_infer.py "
164             f"--video {wsl_tilde_quote(staged_video)} "
165             f"{frame_args}"
166             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167             f"--sport {shlex.quote(self.cfg.sport)} "
168             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv'}}"
169         )
170         logger.info("Running WASB inference on %s ...", base)
171         try:
172             res = self._wsl_bash(cmd)
173         except subprocess.TimeoutExpired:
174             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175             return None
176         if res.returncode != 0:
177             logger.error("WASB inference failed:\n%s", (res.stderr or
res.stdout)[-2000:])
178             return None
179         if not os.path.exists(expected_csv_win):
180             logger.error("WASB finished but expected CSV not found at %s",
expected_csv_win)
181             return None
182         logger.info("WASB trajectory CSV: %s", expected_csv_win)
183
184         # Success ? the trajectory CSV is the durable output; the staged video copy and
185         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
186         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
187         # so the frame cache + detector cache remain for a resumable re-run.
188         # (Streaming never writes a frame cache, so there's only the staged video to
drop.)
189         if not self.cfg.keep_frames:
190             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
staged_video]
191             logger.info("Cache hygiene: removing staged video%s for %s "
192                         "(set indexing.wasb.keep_frames=true to retain).",
193                         "" if self.cfg.stream_video else " + frame cache", key)
194             self._wsl_rm_rf(stale)
195             return expected_csv_win
196
197         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

4. user (2026-06-21T13:56:32.253Z)

```

1     """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3     This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4     of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5     the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.

```

```

6
7 The ABC lets us:
8 - Keep the current WSL adapters working unchanged for Windows users.
9 - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10 the hybrid segmenters or the model-agnostic windowing brain.
11 - Provide a single place to evolve the healthcheck / error contract and thread
12 detector readiness into per-video reports.
13
14 Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15 - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16   Run detector, return the Windows-side path to the produced trajectory CSV
17   (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18   The caller (hybrid) is responsible for output dir; impls may stage the video
19   into WSL fs for perf.
20
21 - healthcheck() -> tuple[bool, str]
22   Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23   Message is logged (and can be surfaced to reports or UX in future).
24   Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26 Existing behavior is 100% preserved. The only additions are the inheritance and
27 the explicit common type for future implementers.
28
29 How to add a new runner (for docs / future contributors):
30 1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31 2. Implement run_predict and healthcheck (return (bool, str) tuple).
32 3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33   TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34   deliberately static and model-agnostic). If the new detector needs different
35   windowing, it can implement its own or we generalize later.
36 4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37   registry never pulls torch/tf on a machine that doesn't have them).
38 5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39   DetectorRunner instances (they already do the right thing via duck-typing + the
40   type hints we add here).
41 6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42   returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43 7. No change to eval/calibration code paths (they consume the CSV + the static
44   windowing helpers directly).
45
46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 ""
68
69 import logging
70 import shlex

```

```

71 import subprocess
72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     `shlex.quote("~/clips/x.mp4")` single-quotes the whole path, so bash
82     treats `~` literally and `cp`/`mkdir` fail with "No such file or
83     directory" ? this broke live WSL staging for the default `~/clips`
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     `~/` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~":
89         return path
90     if path.startswith("~/"):
91         return "~/ " + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared `wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires `self.cfg` with `distro`, `conda_sh`
100     and `timeout_sec` attributes (TrackNetConfig and WasbConfig both qualify).
101     `timeout_sec == 0` maps to None = wait forever (configs default 1800s).
102     """
103
104     cfg: Any
105
106     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109         logger.debug("WSL exec: %s", full)
110         return subprocess.run(argv, capture_output=True, text=True,
111                               timeout=(self.cfg.timeout_sec or None))
112
113     # ---- cache hygiene helpers (shared by both WSL runners) -----
114     def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115         """Best-effort free space (GB) on the WSL filesystem holding `wsl_path`.
116
117         Returns None if it can't be determined (never raises). Used as a pre-run
118         disk guard so a long extraction doesn't fill the disk mid-flight.
119         """
120         try:
121             res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122 'NR==2{{print $4}}'")
123         except (subprocess.TimeoutExpired, OSError):
124             return None
125         try:
126             return int((res.stdout or "").strip()) / (1024 ** 3)
127         except (ValueError, AttributeError):
128             return None
129
130     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
131         """Best-effort recursive delete of WSL paths (post-success cleanup; never
132         raises)."""
133         paths = [p for p in wsl_paths if p]
134         if not paths:
135             return

```

```

134         quoted = " ".join(wsl_tilde_quote(p) for p in paths)
135         try:
136             self._wsl_bash(f"rm -rf {quoted}")
137         except (subprocess.TimeoutExpired, OSError) as e:
138             logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
139
140     def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
141         """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
142
143         Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
144         filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
145         fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
146         trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
147         A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
148         any
149         frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
150         ``cp`` still gets its chance (we never block a genuinely-readable source).
151         """
152         try:
153             res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
154         except (subprocess.TimeoutExpired, OSError):
155             return None
156         if "READABLE" in (res.stdout or ""):
157             return None
158         return (
159             f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
160             f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
161             f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
162         )
163
164     class DetectorRunner(ABC):
165         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
166
167         @abstractmethod
168         def run_predict(self, video_win_path: str, output_win_dir: str,
169                       video_id: Optional[str] = None) -> Optional[str]:
170             """Run inference and return path to trajectory CSV (or None on failure).
171
172             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
173             videos that happen to share a filename cannot reuse each other's frames or CSV.
174             It is optional for back-compat; callers in the live pipeline always pass it.
175             """
176             pass
177
178         @abstractmethod
179         def healthcheck(self) -> Tuple[bool, str]:
180             """Quick check if the runner (env, weights, etc.) is usable.
181
182             Returns (ok, message). Never raises. Cheap to call before a long run.
183             """
184             pass
185
186         # De facto trajectory contract: the trajectory hybrids call these
187         # unconditionally after run_predict. Concrete runners reuse/re-export the
188         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189         # a runner with custom windowing supplies its own with the same shapes.
190         parse_trajectory_csv: Callable[..., Any]
191         trajectory_to_action_windows: Callable[..., Any]
192

```

5. user (2026-06-21T13:56:32.576Z)

```

1     # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2     #

```

```

3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__'d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
37     ]
38
39     [project.optional-dependencies]
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47     ]
48
49     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50     # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51     # fetched over SSH; if you lack access, simply omit this group ? reporting
52     # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53     #   pip install -e ../sports-obsreport --no-build-isolation
54     reporting = [
55         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56     ]
57
58     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60     # express, so they are NOT declarable here. Install them out-of-band instead:
61     #   # CUDA 12.4 box:
62     #   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63     #   # CPU-only box (also what CI uses):
64     #   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65     gpu = []

```

```

66
67     # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68     # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69     # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70     # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71     # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72     storage-s3 = ["boto3>=1.34"]
73     storage-gcs = ["google-cloud-storage>=2.16"]
74
75     [project.scripts]
76     # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77     indexer = "backend.main:main"
78
79     [tool.hatch.metadata]
80     # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81     # reference*); hatchling requires opting in to allow those in metadata.
82     allow-direct-references = true
83
84     [tool.hatch.build.targets.wheel]
85     # `backend` is a namespace package (no __init__.py); list both import roots
86     # explicitly so an editable/wheel install exposes `import backend` and
87     # `import training` regardless of cwd.
88     packages = ["backend", "training"]
89

```

6. assistant (2026-06-21T13:56:35.756Z)

Now let me check the native_wasb_runner.py existence and the khelsutra docs referenced.

7. user (2026-06-21T13:56:36.452Z)

backend\pipeline\detectors\native_wasb_runner.py

8. user (2026-06-21T13:56:36.954Z)

deploy\cloudrun\Dockerfile

9. user (2026-06-21T13:56:37.922Z)

```

1     # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3     > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4     > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5     > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6     > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7     > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9     ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on

```

****one machine****, with `StorageBackend=local` for the MVP. It is the ****single-box deployment profile of the just-shipped decoupled architecture**** ? storage=local + compute=local is the degenerate collapse of the same storage/worker split.

13

14 The ****single most important caveat**** the engine has ****zero app-layer auth and CORS**** ? `[verified backend/main.py:42]`, so safety is structural ****only**** because the engine stays bound to `127.0.0.1:8000` ? `[verified backend/main.py:638]`, a firewall blocks inbound `8000`, and a reverse proxy/tunnel is the sole authenticated ingress ? ****the gate, not the app, is the tenancy boundary.****

15

16 Two honest hardware truths: (1) the current droplet is ****CPU-only**** ? real detection there is ****Gemini (cloud)**** or ****stub (CPU mock)****; ****real local-GPU detection needs a GPU host**** (native WASB, in-flight). (2) The owner priority is to ****stand up a GPU host ASAP**** ? provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs ****in parallel**** with the UI MVP, which ships on Gemini today.

17

18 **## 1. Problem & goal**

19 The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker {infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified backend/main.py]`. What's missing is a ****console**** that lets a non-CLI operator run the whole loop on a single box, ****honestly, behind a gate****.

20

21 ****Headline CUJ (owner's words):**** a ****hosted website**** where I pick an input ? ****a Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload**** ? it lands in the ****bucket (DO Spaces / GCS)**** or locally ? I ****generate highlights**** on a ****GPU-enabled host**** running the real e2e ? I ****pick rallies, watch, and download**** ? ****completely from the UI.****

22

23 ****Outcome ("good")**** an operator signs in, picks a source, watches it index, curates rallies, downloads a reel ? on the ****live CPU droplet today**** (Gemini/stub), and on a ****GPU host later**** (native WASB) with ****no UI fork****.

24

25 ****Read to get current:**** `PER\_RALLY\_SELECTION.md` (\$3/\$4/\$10 contract anchors), engine `HOSTING\_PLAN.md` (gating), `DECOUPLED\_COMPUTE/HANDOFF.md` (what's built), `VIDEO\_LOCALITY\_MODEL.md` (privacy/retention).

26

27 **## 2. Decisions locked**

28 | # | Decision | Source / date | Implication |

29 |---|---|---|---|

30 | D1 | Reverse-proxied ****single origin****; engine stays `127.0.0.1:8000` | `[verified main.py:638]`; 2026-06-20 | CORS ****is moot from the browser ? the proxy is the boundary** |

31 | D2 | ****Edge auth****, zero engine auth code | `HOSTING\_PLAN.md`; 2026-06-20 | Caddy basicauth OR cloudflare + Cloudflare Access |

32 | D3 | MVP drives the ****in-process FastAPI worker**** loop, `storage=local` | `[verified job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |

33 | D4 | Embed ****RallyPicker**** from `PER\_RALLY\_SELECTION.md` ****verbatim**** | PR [#11]; 2026-06-20 | the console is a superset; do not redesign the picker |

34 | D5 | Console lives in the khelsutra ****web/**** React scaffold | `web/README.md`; 2026-06-20 | Cloudflare-Pages-style root, not engine static |

35 | D6 | Compute placement is a ****setting****, rendered from `GET /api/capabilities` | `[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |

36 | D7 | ****Ingest is a source menu**** (local upload · bucket URI · Drive) normalizing to one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a ****stub**** today ? scoped honestly (\$4.2) |

37 | D8 | GPU host is ****provider-agnostic**** (DO droplet / GCE VM / Cloud Run+GPU); storage stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-in |

38 | D9 | ****Observability is first-class**** ? structured logs on every new path + a DoD item | Owner 2026-06-20 | see \$5; verified during CUJ replay |

39 | D10 | Delivery = a ****self-hosted webserver + UI the user spins up**** (NOT a desktop app); the user points at ****local storage + GPU/CPU OR cloud locations**** and the UI drives the tool | Owner 2026-06-20 | ****supersedes `DESKTOP\_APP\_PLAN.md`****; one artifact (this appliance) runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |

40

41 **## 3. Foundation ? what the engine gives us today** `[verified this session unless`

```

noted]`
42     - **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` . `PUT
/api/upload/{id}/part/{i}` . `POST /api/upload/{id}/complete` . `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-
wiring only**, no engine change.
43     - **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)`
`[verified job_worker.py:37]` ? strictly **one job at a time**; the DB `jobs` table is the
clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
**separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1
to serialize) ? it is **not** already 1.
44     - **Storage:** `storage.backend` default = `local` `[verified models.py:298]`;
`LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-
local**. URI scheme `parse_uri` `[verified ref.py]`: bare/`local://` ? local; `s3://`
(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://`/`gs://` ; `gdrive://` ? **follow-up stub
(emulated only)**. Credentials never in config ? boto3 default chain / `~/aws` `[verified
s3.py]`.
45     - **Compute seam:** `indexing.detector_impl` ? `{auto=WSL, native=Linux-native,
`stub`=CPU mock}; default "auto" `[verified models.py:129]`, resolved in
`trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
`feat/native-wasb-runner`:** `native_wasb_runner.py` **exists** `[verified]`;
**fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified
:57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as **not production-
verified**.
46     - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video_id>/{intervals.csv,report.json}` `[verified
worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +
`--set indexing.skip_ai_handoff=true` + `storage.s3`.
47     - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
/api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}`
`[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48     - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run
there.
49     - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51     ## 4. Design / architecture
52
53     ### 4.1 Topology & request path
54     ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI
`127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.
55
56     ...
57     PUBLIC INTERNET
58     ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
59     ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth
60     ?
61     ?????????????????????????????????????????????????????????????????????????????????
62     ? ONE BOX firewall BLOCKS inbound 8000 + 9000/9001/9023 ?
63     ? (2) reverse proxy / static host (Caddy/cloudflared) ?
64     ? /, /assets/* ? web/ React SPA /api/*, /api/highlights/* ? :8000 ?

```

```

65      ?          ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66      ?          ?
67      ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?
68      ?          control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?
69      ?          data: uploads.py chunked upload · /api/compile · /api/highlights ?
70      ?          ?
71      ? (4) STORAGE = LocalStorage (default) fetch=identity · put=no-op(src==dst) ?
72      ?          ?
73      ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet] ?
74      ?          native WASB (GPU host) · auto (owner Win+WSL) ?
75      ?????????????????????????????????????????????????????????????????
76      RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77      ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3://?` ?
outputs/<vid>/ back
78      (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79      ...
80
81      ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82      All sources normalize to "a video the pipeline can read" ? a local path (MVP) or a
`StorageRef` (scale):
83
84      | Source | Path today | Status |
85      |---|---|---|
86      | Local upload | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` [verified uploads.py] ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3://`/`gcs://`.) | wire client only (endpoints exist)
|
87      | Bucket URI (`s3://`, `gs://`) | POST /api/process accepts the bucket URI
directly (resolve ? fetch to scratch ? same pipeline ? API DB) [verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]; the worker CLI also fetches `--video-uri`. So the
engine endpoint is NOT net-new ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | engine: DONE (verified); SPA ingest call: design (= B-P2) |
88      | Google Drive (`gdrive://`) | `parse_uri` recognizes it, but the gdrive backend is
an emulated stub [verified ref.py, DECOUPLED_COMPUTE research] | stub ? roadmap; MVP
requires a Drive?bucket pre-step; do not present Drive as live until the backend is real |
89
90      ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91      | Placement | Runs | Needs | Real/mock | Honest caveat |
92      |---|---|---|---|---|
93      | CPU droplet ? gemini (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; no GPU | REAL | Cloud brain: uploads ?1280px chunks to Google
? must be in consent copy; no per-video cost ceiling today |
94      | CPU droplet ? stub | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | MOCK | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95      | CPU droplet ? motion | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96      | GPU host ? native WASB (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU; `WASB_WEIGHTS` | REAL, on-device | partially wired (fusion builds it, trajectory
refuses); owner byte-parity gate; Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial |
97      | Owner Win+WSL ? auto | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98      | Remote GPU worker via bucket (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a net-new orchestration
endpoint | REAL (remote) | FastAPI loop and worker CLI are separate today; "remote GPU as
a setting" is design ? selector shown disabled in MVP |
99
100     Provider note: Cloud Run + GPU (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own per-video-billing economics (`DECOUPLED_COMPUTE` #246) ? gated on the
containerization feasibility check above.

```

```

101
102     ### 4.4 Auth / exposure (non-negotiable invariant)
103     No engine auth, by design; the gate is external. **Invariant (all three must hold):**
engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
`[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**;
set `security.allowed_video_root`; rate-limit; per-user quota.
104
105     ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106     - **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
`/api/capabilities` ? `POST /api/process`.
107     - **S2 Progress** ? 3s poll of the free-text stage
(`hashing/validating/segmenting/finalizing`), a **"one job at a time"*** notice, and the restart-
failure terminal state.
108     - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`):
honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter.
**Per-rally is ONE screen of the console.**
109     - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for
**reload-survival**.
110     - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target
selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112     ### 4.6 Reuse map
113     **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked
upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
`NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
`HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
`GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the
SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
bridge.
114
115     ## 5. Observability ? logging & monitoring `[required, first-class]`
116     Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously,
with a full log-analysis pass across every new code path.** The engine already uses a
centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new
path.
117
118     - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`)
**plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and
into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ?
GPU host. Log it on every line.
119     - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry
/ result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with
the same correlation id.
120     - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed
chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must
**never** look complete; every dropped clip/chunk is logged + surfaced in the UI.
121     - **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a
thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a
ledger exists) for the engine-mode banner + alerting.
122     - **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is
**ephemeral** ? logs must ship to a durable sink (the bucket or a log service), not die with the
instance.
123     - **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes
a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay**
($8 Layers 173). Ties [[feedback-launch-quality-bar]].
124
125     ## 6. Risk table
126     | Risk | Today | Mitigation |
127     |---|---|---|
128     | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall,
gate} slips | enforce all three; **LIVE posture test** that raw `:8000` is refused from outside
|
129     | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes
them |
130     | Retention | `output/+uploads/` originals **never deleted** | sweep / keep-proxy-only

```

```

before any non-owner exposure |
131 | Surprise Gemini/GPU bill | **no cost ceiling/ledger** | per-video cap;
`ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132 | Native WASB unverified | partially wired; owner-parity-gated | treat as not-
production-verified until owner byte-parity on a real GPU box |
133 | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist |
coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD,
explicit `git add`** |
134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require
Drive?bucket pre-step until real |
135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
136 | Shared-checkout collisions | active worker sessions share the engine tree | the
branch-safety protocol on every engine-side PR |
137 | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted
in chat or committed (cf. the **already-rotated Spaces key** exposed in chat,
`DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive
/ `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token,
revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action
every time** |
138
139 ## 7. Honest limits ? what this does NOT do
140 Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs
global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make
the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142 ## 8. CUJs & live-test plan `[required]`
143 **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145 **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146 - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
link via snapshot/network). Catches reload-survival (CUJ-2), the `play_segments` field-name bug
(CUJ-3), capability honesty (CUJ-4).
147 - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main
--server` with `storage=local` + `detector_impl=stub` + `skip_ai_handoff` and assert the **mock
and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play_segments` schema,
compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock
handler **and** a real-engine contract test **in the same PR**. Runs in CI.
148 - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1
with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety
posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption;
reuse the `parity.yml` dual-host pattern.
149 - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job +
headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no
stacking**; coverage at/above the floor; **logs inspected during replay** ($5).
150 - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU
box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/cv2-vs-ffmpeg`
behavior; a multi-GB browser upload. Reported back into $9, stated plainly as **not verified**
in the offline suite.
151
152 ## 9. Deliverables & progress tracker ? **source of truth**
153 Legend: ? Todo . ? In progress . ? Done . ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR
ships its tests + meaningful logs ($5).

```

```

154
155 | ID | Deliverable | Depends on | Gated? | Status | PR |
156 |----|-----|-----|-----|-----|----|
157 | T0 | This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers | ? | owner | ? |
(this PR) |
158 | T1 | Mock engine + capability fixtures + browser-replay CUJ scripts (L1) | T0 | no | ? |
| ? |
159 | T2 | Appliance shell: Caddyfile + systemd units; firewall; **edge auth**; rotate key
(P1) | ? | owner | ? | ? |
160 | T3 | L3 live posture test (raw `:8000`/`:900x` refused from outside) | T2 | no | ? | ? |
|
161 | T4 | S1 ingest (local upload wired) + segmenter picker | T1 | no | ? | ? |
162 | T5 | S2 progress (poll + one-job + restart-failure) | T4 | no | ? | ? |
163 | T6 | S3 embed **RallyPicker** + compile + download | T4 | no | ? | ? |
164 | T7 | *(engine)* `GET /api/capabilities` + `GET /api/health` + banner | T1 | coord | ? |
| ? |
165 | T8 | *(engine)* `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival) |
? | coord | ? | ? |
166 | T9 | S5 Settings (config+capabilities; disabled compute/storage target) | T7,T8 | no |
? | ? |
167 | T10 | Observability pass: correlation id end-to-end + structured logs + metrics (§5) |
T4 | no | ? | ? |
168 | T11 | Retention sweep for `output/`+`uploads/` | ? | owner | ? | ? |
169 | T12 | Ingest from **bucket URI** (`s3://`/`gs://`) wiring ? **engine half DONE**
(`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (=
**B-P2**) | T4 | coord | ? | engine done; SPA pending |
170 | T13 | **GPU host stand-up** (provider-agnostic) + wire `detector_impl=native`; owner
byte-parity (P4) | branch | ? owner | ? | ? |
171 | T14 | *(reserved)* Bucket-worker bridge ? remote-GPU "compute as a setting" (P5) |
T8,T12 | ? owner | ? | ? |
172
173 **Owner priority (2026-06-20): T13 (GPU host) is elevated to run in parallel from the
start** ? owner provisions the host + token; the engine session lands native WASB; the UI MVP
does **not** block on it (ships on Gemini).
174
175 ## 10. Open questions ? owner / external
176 **Blocking gates:** (1) **edge-auth** ? Caddy basicauth (self-contained) vs
cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention
defaults** ? auto-delete originals after compile? keep proxies+intervals only? window?
(privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4)
**GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker
worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration
endpoint + how it coordinates with the active worker sessions.
177 **Nice-to-knows:** console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.
178
179 ## 11. Definition of done
180 - [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via
the ingest menu.
181 - [ ] Raw `:8000`/`:900x` unreachable from outside (**tested**, not assumed).
182 - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).
183 - [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
184 - [ ] `api/capabilities` + `api/health` + banner truthful; reload-survival via
`api/videos` + `api/reels`.
185 - [ ] **Observability:** correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.
186 - [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
187 - [ ] CI green (L2 contract + L4 regression); coverage ? floor; §9 tracker all
??-resolved.
188
189 ## 12. References
190 **Engine code `[verified this session]`:**
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;

```

```
`backend/api/static/app.js:88,344,373`;
`backend/storage/{base.py, local.py:33, ref.py, __init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:
gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
`DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
`DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
`PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,
`PORTABILITY.md`, `web/README.md`.
```

191

192 ---

193 ### Changelog

194 - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs
(FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin
spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor
re-verified against source before writing (catching the native-WASB partial-wiring +
`ai\_max\_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local),
multi-provider GPU, and first-class Observability directives.

195

10. assistant (2026-06-21T13:56:41.406Z)

Interesting ? there IS a Dockerfile at `deploy/cloudrun/Dockerfile`, which directly contradicts
claim 12 (the "no Dockerfile" claim). Let me examine it and the native_wasb_runner.py.

11. user (2026-06-21T13:56:42.319Z)

```
1      # Cloud Run GPU **Job** image for the rally worker (COMPUTE_DECOUPLED_SERVING M6/M7,
ADR-012/D16).
2      #
3      # Runs `python -m backend.worker infer ?` INLINE on an L4 (detect + window + stitch co-
located so
4      # CPU/wall + egress are metered into one job ? D4). Built + pushed + verified by the
owner on the
5      # M7 real run (the $100/?10k budget, D17); CI does NOT build this (a CUDA image is too
heavy) ? the
6      # dispatch shim is proven with a fake client (tests/test_compute_backend.py).
7      #
8      # Two python stacks, deliberately separate (a framework conflict otherwise ? same split
as the
9      # native GPU box, deploy/digitalocean/gpu_setup.sh):
10     # * the MAIN app env (this repo + ffmpeg) drives the pipeline + stitching;
11     # * a conda `wasb` env (py3.8 + torch 1.11.0+cu113) runs the WASB shuttle detector as
a subprocess
12     # (the native runner shells out to $WASB_PYTHON), so the app's torch never co-
installs with it.
13     #
14     # ? WASB-C3: the pretrained badminton weights are academic-data-trained ? provenance NOT
cleared for
15     # commercial sharing. They are NOT baked into this image; stage them at runtime (a
bucket fetch or a
16     # mount) and point $WASB_WEIGHTS at them. Resolve WASB-C3 before any public, non-owner
serving.
17
18     FROM nvidia/cuda:12.4.1-runtime-ubuntu22.04
19
20     ENV DEBIAN_FRONTEND=noninteractive \
21         PYTHONUNBUFFERED=1 \
22         PIP_NO_CACHE_DIR=1 \
23         CONDA_DIR=/opt/conda \
24         WASB_REPO_DIR=/opt/models/WASB-SBDT
25
26     # ffmpeg (stitch/decode) + DejaVu fonts (the watermark filter) + git/curl for the WASB
```

```

env build.
27     RUN apt-get update && apt-get install -y --no-install-recommends \
28         ffmpeg fonts-dejavu-core git curl ca-certificates build-essential \
29         python3 python3-pip python3-venv \
30         && rm -rf /var/lib/apt/lists/*
31
32     # --- WASB detector env (mirrors deploy/digitalocean/gpu_setup.sh ? the verified
torch-1.11 stack) ---
33     RUN curl -sSL https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh -o
/tmp/mc.sh \
34         && bash /tmp/mc.sh -b -p "$CONDA_DIR" && rm -f /tmp/mc.sh \
35         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/main || true \
36         && "$CONDA_DIR/bin/conda" tos accept --override-channels --channel
https://repo.anaconda.com/pkgs/r || true \
37         && "$CONDA_DIR/bin/conda" create -y -q -n wasb python=3.8 \
38         && git clone --depth 1 https://github.com/nttcom/WASB-SBDT.git "$WASB_REPO_DIR" \
39         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
40             torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113 \
41         && ( [ -f "$WASB_REPO_DIR/requirements.txt" ] && "$CONDA_DIR/envs/wasb/bin/python"
-m pip install -q -r "$WASB_REPO_DIR/requirements.txt" || true ) \
42         && "$CONDA_DIR/envs/wasb/bin/python" -m pip install -q \
43             hydra-core omegaconf pandas opencv-python-headless pillow scipy tqdm pyyaml
matplotlib "numpy<1.24"
44
45     # --- Main application env ---
46     WORKDIR /app
47     COPY pyproject.toml README.md ./
48     COPY backend ./backend
49     COPY training ./training
50     RUN python3 -m pip install --upgrade pip && python3 -m pip install .
51
52     # The native runner finds the WASB env + repo by env (no `conda activate` needed);
weights come in at
53     # runtime (WASB-C3). detector_impl=native = the Linux-native WASB runner; cuda stays the
default.
54     ENV WASB_PYTHON=/opt/conda/envs/wasb/bin/python \
55         WASB_REPO_DIR=/opt/models/WASB-SBDT \
56         WASB_WEIGHTS=/opt/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar \
57         RALLY_DETECTOR_IMPL=native
58
59     # The dispatcher (CloudRunCompute) supplies the per-execution args:
60     # infer --video-uri gs:/// --out-uri gs:/// --done-uri gs:/// --segmenter wasb_hybrid
--set ?
61     ENTRYPOINT ["python3", "-m", "backend.worker"]
62

```

12. user (2026-06-21T13:56:42.745Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8     see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10     * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked by path), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.

```

```

13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
14     ``auto``
15     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
16     ``cuda`` stays
17     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
18     ``runner.device`` override).
19     * weights / repo / python / scratch come from the **environment** first
20     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
21     ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
22     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
23     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
24     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
25     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
26     (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
27     ``detector_impl="native"``.
28     **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
29     runner
30     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
31     same
32     candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
33     deterministic
34     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
35     exact
36     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
37     streaming
38     vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
39     artifact), and two native runs were byte-identical (deterministic). This is a hardware
40     check
41     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
42     contract,
43     not the numbers.
44     """
45     import os
46     import shutil
47     import logging
48     import subprocess
49     from dataclasses import dataclass
50     from typing import Any, Dict, List, Optional, Tuple
51
52     from backend.pipeline.detectors.base import DetectorRunner
53     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
54     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
55     WSL/TrackNet.
56     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
57
58     logger = logging.getLogger(__name__)
59
60     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
61     # detectors/trackers/data loaders + find configs/), exactly as the WSL adapter does.
62     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
63
64     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
65     probe
66     # so the "cuda True/False" string they parse can never drift apart.
67     _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',
68     torch.cuda.is_available())"
69
70     @dataclass
71     class NativeWasbConfig:
72         """Resolved settings for a Linux-native WASB run.
73
74         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence

```

```

64         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
``device``
65         defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
perf win).
66         """
67         repo_dir: str = "~/models/WASB-SBDT"      # native clone of nttcom/WASB-SBDT
68         python_bin: str = "python"                # the `wasb` conda env python (invoked by
path, no activate)
69         weights_path: str = ""                    # REQUIRED (env WASB_WEIGHTS or
indexing.wasb.weights_path)
70         sport: str = "badminton"
71         device: str = "cuda"                       # auto | cuda | mps | cpu (auto = cuda-if-
available-else-cpu)
72         scratch_dir: str = ""                     # frame-extraction scratch (env); "" = next
to the output dir
73         timeout_sec: int = 1800                    # 30 min; a hung GPU call is killed (0 = no
timeout)
74         keep_frames: bool = False                  # retain the decoded frame cache after
success (debug only)
75         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
clip's
76         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
real GPU
77         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
here.
78         stream_video: bool = True
79
80     @classmethod
81     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
82         w = (idx_cfg or {}).get("wasb", {}) or {}
83         env = os.environ.get
84         # stream_video is tri-state in config: None/absent => native default (stream);
an
85         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
86         sv = w.get("stream_video")
87         return cls(
88             repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
89             python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
90             weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
91             sport=w.get("sport", "badminton"),
92             device=env("WASB_DEVICE") or w.get("device", "cuda"),
93             scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
""",
94             timeout_sec=int(w.get("timeout_sec", 1800)),
95             keep_frames=bool(w.get("keep_frames", False)),
96             stream_video=(True if sv is None else bool(sv)),
97         )
98
99
100     class NativeWasbRunner(DetectorRunner):
101         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
docstring."""
102
103         def __init__(self, cfg: NativeWasbConfig):
104             self.cfg = cfg
105             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
healthcheck;
106             # lazily probed by run_predict if healthcheck was skipped. None = not yet
probed.
107             self._cuda_available: Optional[bool] = None
108
109             # --- low-level native exec (the single place tests patch)
110             -----
111             def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:

```

```

111         logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
112         return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
113                               timeout=(self.cfg.timeout_sec or None))
114
115     # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
116     def _probe_cuda(self) -> bool:
117         """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
118         (missing python / timeout / import error) is treated as 'no CUDA'."""
119         try:
120             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
121         except (FileNotFoundError, subprocess.TimeoutExpired):
122             return False
123         return res.returncode == 0 and "cuda True" in (res.stdout or "")
124
125     def _cuda_is_available(self) -> bool:
126         """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
127         if self._cuda_available is None:
128             self._cuda_available = self._probe_cuda()
129         return self._cuda_available
130
131     def _effective_device(self) -> str:
132         """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
133         probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
134         if self.cfg.device == AUTO:
135             return DeviceContext(AUTO, self._cuda_is_available()).effective
136         return self.cfg.device
137
138     def _repo_src(self) -> str:
139         return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
140
141     def _copy_infer_script(self) -> bool:
142         """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
143         repo_src = self._repo_src()
144         try:
145             os.makedirs(repo_src, exist_ok=True)
146             shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
147             return True
148         except OSError as e:
149             logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
150             return False
151
152     def _rm_rf(self, path: Optional[str]) -> None:
153         """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
154         if path:
155             shutil.rmtree(path, ignore_errors=True)
156
157     def healthcheck(self) -> Tuple[bool, str]:
158         """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
159         if not self.cfg.weights_path:
160             return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
161         weights = os.path.expanduser(self.cfg.weights_path)
162         if not os.path.exists(weights):
163             return False, f"WASB weights not found at {weights}"
164         repo_src = self._repo_src()
165         if not os.path.isdir(repo_src):
166             return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone

```

```

nttcom/WASB-SBDT "
167             f"(set WASB_REPO_DIR / indexing.wasb.repo_dir.)")
168         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
169         # argparse rc=2 from wasb_infer.py at run time.
170         if self.cfg.device not in VALID_DEVICES:
171             return False, (f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)).}")
172         try:
173             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
174         except FileNotFoundError:
175             return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
176         except subprocess.TimeoutExpired:
177             return False, "native torch healthcheck timed out."
178         out = (res.stdout or "").strip()
179         if res.returncode != 0:
180             return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
181         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
182         self._cuda_available = "cuda True" in out
183         ctx = DeviceContext(self.cfg.device, self._cuda_available)
184         if ctx.cuda_required_but_missing:
185             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
186             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
187             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
188         if ctx.fell_back:
189             logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
CPU; WASB "
190                             "inference will be slow. Set indexing.wasb.device=cuda to
require a GPU.", out)
191         return True, out
192
193     def run_predict(self, video_win_path: str, output_win_dir: str,
194                    video_id: Optional[str] = None) -> Optional[str]:
195         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
196
197         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
198         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
199         same downstream contract, so the parity comparison is apples-to-apples.
200         """
201         output_dir = os.path.abspath(output_win_dir)
202         os.makedirs(output_dir, exist_ok=True)
203         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
204         norm = str(video_win_path).replace("\\", "/")
205         stem, _ext = os.path.splitext(os.path.basename(norm))
206         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
207         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
208         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
209
210         if not self._copy_infer_script():
211             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
212             return None
213
214         weights = os.path.expanduser(self.cfg.weights_path)
215         argv = [self.cfg.python_bin, "wasb_infer.py",
216                "--video", os.path.abspath(str(video_win_path))]
217         frames_dir: Optional[str] = None
218         if self.cfg.stream_video:

```

```

219         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
220         argv.append("--stream-video")
221     else:
222         scratch = self.cfg.scratch_dir or output_dir
223         frames_dir = os.path.join(scratch, f"{key}_frames")
224         argv += ["--frames_out_dir", frames_dir]
225         # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
226         # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
227         device = self._effective_device()
228         argv += ["--weights", weights,
229                 "--sport", self.cfg.sport,
230                 "--device", device,
231                 "--out", out_csv]
232
233         logger.info("Running native WASB inference (device=%s) on %s ...",
234                     device, " [auto?fallback]" if self.cfg.device == AUTO and device !=
"cuda" else "",
235                     os.path.basename(norm))
236         try:
237             res = self._run(argv, cwd=self._repo_src())
238         except FileNotFoundError:
239             logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
240             return None
241         except subprocess.TimeoutExpired:
242             logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
243             return None
244         if res.returncode != 0:
245             logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
246             return None
247         if not os.path.exists(out_csv):
248             logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
249             return None
250         logger.info("WASB trajectory CSV: %s", out_csv)
251
252         # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
253         # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
254         # (Streaming never materializes frames, so there is nothing to drop on that
path.)
255         if not self.cfg.keep_frames and frames_dir:
256             logger.info("Cache hygiene: removing frame cache for %s "
257                         "(set indexing.wasb.keep_frames=true to retain).", key)
258             self._rm_rf(frames_dir)
259         return out_csv
260
261         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
262         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
263         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
264

```

13. assistant (2026-06-21T13:56:48.570Z)

Significant findings already. The native_wasb_runner.py module docstring says it was "? Validated 2026-06-20 on an RTX 2070" ? which contradicts claim 1's "not production-verified." But the khelsutra doc says "owner byte-parity gate" not yet passed. There's nuance. Also

crucially: the native runner does NOT eliminate the second torch env ? it still shells out to a separate `wasb` conda env python (py3.8 + torch 1.11). That directly undercuts the recommendation's central premise. Let me check git state and the recent commits about M6/M7, and verify the "no Dockerfile" claim against the khelsutra doc's update date.

14. user (2026-06-21T13:56:57.163Z)

```
8cdf65c feat(serving): M6 ? Cloud Run GPU dispatch shim + worker image (default-OFF, mocked)
(#287)
---BRANCH---
feat/gcp-ops-agent-and-dryrun-guide
---DOCKERFILE EXISTS---
total 8
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:20 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:11 ..
-rw-r--r-- 1 avidu 197609 3716 Jun 21 16:11 Dockerfile
-rw-r--r-- 1 avidu 197609 3927 Jun 21 16:20 README.md
```

15. user (2026-06-21T13:56:58.573Z)

```
7750afc docs(dry-run): Path-B live e2e DONE ? in-UI bucket?reel CUJ + screencast (DoD $6 box 3
?) (#57)
4525c5d feat(web): B-P2 ? in-UI bucket ingest (point at a gs:// video ? process ? load) (#56)
d0461fd docs: correct stale "bucket ingest is net-new" claims (B-P1 already shipped in M2/#280)
(#55)
2421557 docs(dry-run): record Path-A local result (en+hi reel downloaded) + the engine fix it
surfaced (#54)
113587f docs: set the dry-run CUJ as a tracked goal (UI drives a bucket video -> reel download,
multilingual) (#53)
---LAST MODIFIED LOCAL---
2026-06-21 15:05:45 +0530 docs: correct stale "bucket ingest is net-new" claims (B-P1 already
shipped in M2/#280) (#55)
```

16. assistant (2026-06-21T13:57:03.627Z)

This confirms a critical timeline issue. The Dockerfile at `deploy/cloudrun/Dockerfile` was created Jun 21 16:11 (commit #287, M6), but the khelsutra doc `LOCAL\_APPLIANCE\_ARCHITECTURE.md` was last modified Jun 21 15:05 (#55) and dated 2026-06-20. So the "no Dockerfile" claim (claim 12) was true when written but is now STALE ? a Dockerfile now exists and it bundles ffmpeg. Let me also verify the D4 (ffmpeg LGPL) reference and the D10 line number claim.

17. user (2026-06-21T13:57:05.798Z)

```
7750afcd3e5ecd14852d99433da6f479a181a073 2026-06-21 19:04:57 +0530
---uncommitted?---
no output above = committed/clean
```

18. user (2026-06-21T13:57:07.172Z)

```
C:\Users\avidu\Projects\khelsutra\docs\LOCAL_APPLIANCE_ARCHITECTURE.md:33:| D4 | Embed
**RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim** | PR [#11]; 2026-06-20 | the console
is a superset; do not redesign the picker |
C:\Users\avidu\Projects\khelsutra\docs\FRIENDLY_AND_MULTILINGUAL.md:27:| D4 | **Engine P2
backend-localization stays owner-gated + cross-repo** (not in this DoD) | `CLAUDE.md` guardrails
| Dynamic engine error/report text localizes later; tracked as a deferred row |
C:\Users\avidu\Projects\khelsutra\docs\ANNOTATE_AND_TRAIN.md:23:| D4 | NO "Train now"
button/endpoint in the SPA | `[verified]` training is offline `backend/worker train` | SPA's
honest role = correction loop ? grow corpus |
C:\Users\avidu\Projects\khelsutra\docs\PER_RALLY_SELECTION.md:29:| D4 | Compile via existing
`POST /api/compile {video_id, segment_ids, output_filename}` | `[verified
backend/main.py:342-396]` | Selection order is harmless ? stitcher `_merge_overlapping` de-dupes
`[verified compiler.py:80-105, 311-317]` |
```

19. assistant (2026-06-21T13:57:11.032Z)

Important finding: The recommendation references "D4 (no GPL x264)" for ffmpeg, but D4 in `LOCAL\_APPLIANCE\_ARCHITECTURE.md` is "Embed RallyPicker verbatim" ? NOT about ffmpeg/x264 licensing. The "D4" the recommendation cites does not exist as described. Let me search for the actual ffmpeg licensing decision across the khelsutra and engine docs.

20. user (2026-06-21T13:57:32.662Z)

README.md:315:[Omitted long matching line]
docs\VIDEO_LOCALITY_MODEL.md:131:| Q05 | **Windows client packaging** ? portable ffmpeg + a signed tiny app vs a PowerShell script for the pilot | pilot can be a script the associate/owner runs; product needs an installer story (ffmpeg LGPL build = licensing-clean) |
docs\DESKTOP_APP_PLAN.md:38:| D4 | **Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only** (code, weights). ffmpeg ships as an **LGPL build** (no GPL-only encoders like x264). | `COMMERCIALIZATION` (ratified 2026-06-09); `VIDEO\_LOCALITY\_MODEL` Q05 (**"ffmpeg LGPL build = licensing-clean"**) | WASB-SBDT weights are MIT `[verified]` (see `tracknet-wasb-wsl2-decision`). The stitch must use openh264 (BSD) or OS hardware encoders, **not** statically-linked x264. See §3.4. |
docs\DESKTOP_APP_PLAN.md:45:- **WASB on a discrete laptop GPU is acceptable** `[verified, measured]`: on the owner's **RTX 2070 Super Max-Q (8 GB)** , WASB inference is the pipeline driver at **~3.273.4x real-time** (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.776.0 GB; proxy (NVENC) ~0.2x RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5x clip length. Source: `current-state.md` `COST\_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
docs\DESKTOP_APP_PLAN.md:64:- **ffmpeg licensing (a genuine landmine):** ffmpeg is LGPL/GPL. The current stitch path uses **libx264 (GPL)** . Redistributing that inside an installer would impose GPL on the bundle. **Mitigation (D4):** ship an **LGPL ffmpeg build** using **openh264 (BSD)** or the OS hardware encoder (NVENC / Apple **VideoToolbox** / Linux **VA-API**) ? the desktop app should prefer hardware H.264 anyway (faster, no CPU stitch tax).
docs\DESKTOP_APP_PLAN.md:111:| ffmpeg GPL contamination via libx264 in the installer | high (legal) | D4: LGPL build + openh264/hardware encoders only. |
docs\DESKTOP_APP_PLAN.md:133:| **P3** | **Packaging + code-signing installers** per OS (Win MSIX/Authenticode, mac .dmg + Developer-ID notarization, Linux AppImage/Flatpak); LGPL ffmpeg bundling; MIT/Apache/BSD audit of the bundle. | P2 | Yes | ? | ? |
docs\DESKTOP_APP_PLAN.md:152:- [] **P3** signed/notarized installers exist for the greenlit OSes; the bundle passes an MIT/Apache/BSD (+ LGPL-ffmpeg) license audit.
docs\DESKTOP_APP_PLAN.md:159:**External (to validate at P0/P2/P3):** PyTorch MPS backend · onnxruntime (CPU/CoreML/DirectML execution providers) · Tauri vs Electron sizing · ffmpeg LGPL build / openh264 · Apple Developer-ID notarization · Windows Authenticode/MSIX.
docs\COMPUTE_DECOUPLED_SERVING\README.md:43:[Omitted long matching line]
docs\COMPUTE_DECOUPLED_SERVING\README.md:70:3. **STITCH (CPU/ffmpeg):**
`VideoCompiler.compile\_highlights(raw\_video, segments, out\_name) ? reel`
(`stitcher/compiler.py:303`). Pure FS+subprocess (merge ? fprobe ? watermark ? per-clip libx264 trim ? concat). Deterministic for a given input + ranges + watermark config.
docs\COMPUTE_DECOUPLED_SERVING\README.md:142:8. ? **Codec ? D16:** **libx264 ENCODE** (protect parity) + **NVDEC DECODE-only** (speed, no output-byte change); NVENC-encode deferred behind a config knob.
docs\COMPUTE_DECOUPLED_SERVING\README.md:164:[Omitted long matching line]
docs\COMMERCIALIZATION.md:35:| C5 | **Output codec royalties** | **P0** | ? | Highlights are still encoded **H.264 + AAC** (`libx264` in `backend/pipeline/stitcher/compiler.py`) ? encode-side AVC patent exposure remains. | Switch output to **AV1 + Opus** (royalty-free) on an **Ada-class GPU (L4/L40)** for HW encode. See §4. Decision: provision Ada-class GPU? |
docs\COMMERCIALIZATION.md:69:| ffmpeg/fprobe, google-genai, torch, opencv, numpy, fastapi/uvicorn/pydantic/jinja2/dotenv | LGPL/Apache/BSD/MIT | No | ? CLEAR | Invoked as subprocess / imported; all permissive. |
docs\COMMERCIALIZATION.md:100:**The trap:** ffmpeg being free (LGPL) does **not** grant the *patent* rights to the
docs\COMMERCIALIZATION.md:107: (royalty-free)**. NVENC AV1 HW-encode needs **Ada-class GPUs (L4/L40)**; Ampere/Turing
docs\CODE_MAP.md:67: per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy -> output/<name>.mp4
docs\archives\past_projects\DECOUPLED_COMPUTE\07-COMPUTE-ABSTRACTION.md:48:| ffmpeg chunk extraction / stitching | **CPU** (`libx264`); GPU optional (`nvenc`) | no driver-codec dependency by default |
docs\archives\past_projects\DECOUPLED_COMPUTE\06-RISKS-AND-OPEN-QUESTIONS.md:34:5. **Data-contract break via re-encode.** MD5 keys the whole 3-repo join; the collector's
`nvenc`/`libx264`

```
docs\archives\past_projects\DECOUPLED_COMPUTE\02-COMPUTE-MAP-AND-CONTRACT.md:17:| **ffmpeg`
chunking / stitching** | CPU (`libx264`) | **LOW.** No nvenc/driver-codec dependency (a
portability win) ? `apt-get install ffmpeg` is enough. |
docs\archives\past_projects\DECOUPLED_COMPUTE\02-COMPUTE-MAP-AND-CONTRACT.md:35:for proxy
transcode + batch transcode ? defaults to CPU `libx264`. Shares the same local-filesystem + MD5
docs\archives\past_projects\DECOUPLED_COMPUTE\01-EXISTING-PLANS-AND-GAP.md:95:gap is narrow":
its only GPU touchpoint is opt-in `nvenc` (defaults to CPU `libx264`); it shares the same
docs\archives\MONETIZATION_AUDIT.md:28:| **ffmpeg / ffprobe** | LGPL-2.1+ (GPL if `--enable-
gpl`); **not AGPL** | **No** ? no distribution, no network clause | ? CLEAR | Keep; invoke as
subprocess (already do) |
docs\archives\MONETIZATION_AUDIT.md:31:| **opencv-python** | MIT (pkg) / Apache-2.0 (OpenCV) /
bundled ffmpeg is **LGPL** | No | ? CLEAR | Keep |
docs\archives\MONETIZATION_AUDIT.md:89:**The trap:** ffmpeg being free (LGPL) does **not** grant
you the *patent* rights to the codecs it implements. ffmpeg's own legal page explicitly
disclaims patent licenses. Patent royalties attach to the **act of encoding/decoding**,
independent of the software license.
docs\archives\MONETIZATION_AUDIT.md:96: - **GPU acceleration:** NVENC **AV1 encode requires
Ada-class** GPUs (RTX 40, **L4 / L40**). Ampere (A10/A100) and Turing (T4) can *decode* AV1 but
**cannot HW-encode** it ? use **SVT-AV1 (CPU)** on those, while H.264/HEVC encode stays HW-
accelerated everywhere.
docs\archives\MONETIZATION_AUDIT.md:122:- ffmpeg legal: `ffmpeg.org/legal.html`. Codec pools:
Via LA (`via-la.com`), Wikipedia H.264 / HEVC patent-licensing. NVIDIA NVENC matrix:
`developer.nvidia.com/video-encode-and-decode-gpu-support-matrix`. SVT-AV1: AOMedia repo
(GitLab).
docs\archives\decisions\DECISIONS.md:246:4. **Emit royalty-free output.** Standardize highlight
output on **AV1 (+ Opus)** to avoid AVC/HEVC encode-side patent exposure; provision an **Ada-
class GPU (L4/L40)** for hardware AV1 (NVENC), falling back to **SVT-AV1 (CPU, BSD-3-Clause-
Clear)** on T4/A10/A100.
docs\archives\decisions\DECISIONS.md:714:[Omitted long matching line]
deploy\cloudrun\README.md:61:- libx264 encode (parity, D16) + NVDEC decode-only; the image keeps
ffmpeg's default libx264.
backend\utils\video.py:26: reencode: If True, re-encodes with libx264 (slower but frame-
accurate).
backend\utils\video.py:42: cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf",
"23", "-c:a", "aac"]
backend\tools\rally_slicer.py:142: "-c:v", "libx264", "-preset", "superfast", "-crf",
"22",
backend\pipeline\stitcher\compiler.py:253: "-c:v", "libx264",
C:\Users\avidu\Projects\khesutra-screencast\node_modules\playwright-
core\lib\coreBundle.js:25977: "libx264-155",
C:\Users\avidu\Projects\khesutra-screencast\node_modules\playwright-
core\lib\coreBundle.js:26036: "libx264.so": "libx264-155",
C:\Users\avidu\Projects\khesutra-screencast\node_modules\playwright-
core\lib\coreBundle.js:26239: "libx264-163",
C:\Users\avidu\Projects\khesutra-screencast\node_modules\playwright-
core\lib\coreBundle.js:26330: "libx264.so": "libx264-163",
C:\Users\avidu\Projects\khesutra-screencast\node_modules\playwright-
core\lib\coreBundle.js:26452: "libx264-164",
```

[Showing results with pagination = limit: 40]

21. assistant (2026-06-21T13:57:39.144Z)

Critical finding: the "D4 (no GPL x264)" reference is actually from `DESKTOP\_APP\_PLAN.md:38` ? which is the document that D10 EXPLICITLY SUPERSEDES per claim 10. So the recommendation cites a decision (LGPL ffmpeg, openh264/NVENC, no x264) from a superseded plan, AND the current production code (`compiler.py:253`, `rally\_slicer.py:142`, `video.py:42`) actively uses libx264 (GPL encoder). The Cloud Run Dockerfile README even notes it "keeps ffmpeg's default libx264." So the LGPL/openh264 build is aspirational, not current state.

Now let me verify D10's exact line. The doc shows D10 at line 39 of the table. Now let me move to the web-checkable external claims. Let me load WebSearch/WebFetch.

22. assistant (2026-06-21T13:57:44.022Z)

Now let me verify the external claims. Starting with the uv PyTorch claim (claim 2) and Pinokio (claim 3).

23. user (2026-06-21T13:57:50.401Z)

UV Support for PyTorch Index Configuration

Yes, uv fully supports both capabilities you described.

PyTorch-Specific Index Configuration

You can configure PyTorch indexes in `pyproject.toml` using `[[tool.uv.index]]` with the `explicit = true` setting:

```
``toml
[[tool.uv.index]]
name = "pytorch-cpu"
url = "https://download.pytorch.org/whl/cpu"
explicit = true
``
```

The documentation states: `"We recommend the use of explicit = true to ensure that the index is _only_ used for torch, torchvision, and other PyTorch-related packages"`

Routing via tool.uv.sources

You can route `torch` and `torchvision` to specific backends using `[tool.uv.sources]`:

```
``toml
[tool.uv.sources]
torch = [
  { index = "pytorch-cpu" },
]
torchvision = [
  { index = "pytorch-cpu" },
]
``
```

CPU vs CUDA Backend Selection

The guide demonstrates selecting between different backends (CPU-only, CUDA 11.8, 12.6, 12.8, 13.0, ROCm, Intel GPUs) by configuring separate indexes pointing to `download.pytorch.org/whl/{backend}` URLs and using environment markers:

```
``toml
torch = [
  { index = "pytorch-cpu", marker = "sys_platform != 'linux'" },
  { index = "pytorch-cu130", marker = "sys_platform == 'linux'" },
]
``
```

24. user (2026-06-21T13:57:52.265Z)

Pinokio License and Setup Claims

****License:**** MIT license

****Setup Claims:**** The README does not explicitly state that Pinokio installs Python, Conda, virtual environments, GPU libraries, or FFMPEG.

Regarding dependency management, the documentation notes: "the `venv` attribute is included, so the python dependencies are also installed in an app-isolated manner."

It also mentions "The built-in package mangagers (Conda, Homebrew, Pip, and NPM) install everything inside the isolated pinokio home path (`~/pinokio`) by default."

However, the provided content does not describe launching a local web UI or specify which dependencies are automatically installed. The README focuses on security architecture and script verification rather than detailing the complete list of pre-installed components.

25. assistant (2026-06-21T13:57:56.761Z)

Claim 2 (uv) is confirmed solid. Claim 3 (Pinokio) is partially confirmed: license is MIT, and it does isolated env management with Conda/Pip ? but the exact marketing wording about "installs Python, Conda... GPU libraries... installs the latest version of FFMPEG... launches a local web UI" was NOT found on the GitHub README. That specific phrasing is likely from pinokio.computer (the website), not the GitHub repo cited. Let me verify the exact wording on the Pinokio website.

26. user (2026-06-21T13:58:04.259Z)

Ollama Windows Installer - Key Details

****Administrator Rights:**** Yes, it installs without them. "The Ollama install does not require Administrator, and installs in your home directory by default."

****Approximate Size:**** The documentation specifies "at least 4GB of space for the binary install" but doesn't state the installer's own size.

****Background Service:**** Yes, it runs in the background. "Ollama will run in the background and the `ollama` command line is available in `cmd`, `powershell`."

****GPU Auto-Detection:**** The documentation doesn't address auto-detection. It lists requirements ("NVIDIA 452.39 or newer Drivers if you have an NVIDIA card") but doesn't explain whether detection is automatic.

****Model Storage Location:**** It uses `%HOMEPATH%\ollama`. The docs state: "explorer %HOMEPATH%\ollama contains models and configuration" and note this can be changed via the `OLLAMA\_MODELS` environment variable.

****Offline Operation:**** The documentation doesn't clarify whether it runs offline after installation.

****Update Process:**** The installer handles updates. "We update Ollama regularly to support the latest models, and this installer will help you keep up to date," though the exact update mechanism isn't detailed.

27. user (2026-06-21T13:58:10.043Z)

Web search results for query: "Pinokio "installs the latest version of FFMPEG" "Python, Conda" GPU libraries one-click"

Links: [{"title": "Pinokio - Download - Softpedia", "url": "https://www.softpedia.com/get/System/Pinokio.shtml"}, {"title": "Pinokio won't install FFmpeg - ERROR conda.core.link:_execute(938): post-link script failed for package conda-forge · Issue #910 · pinokiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/issues/910"}, {"title": "Releases · pinokiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/releases"}, {"title": "GitHub - 6Morpheus6/pinokio-wiki: Tutorial for Pinokio and its Applications · GitHub", "url": "https://github.com/6Morpheus6/pinokio-wiki"}, {"title": "Installs an outdated version of ffmpeg · Issue #1052 · pinokiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/issues/1052"}, {"title": "program.pinokio", "url": "https://program.pinokio.computer/"}, {"title": "Can't Install this FFMPEG · Issue #908 · pinokiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/issues/908"}, {"title": "Pinokio (software) ? Grokipedia", "url": "https://grokipedia.com/page/Pinokio_software"}, {"title": "Pinokio", "url": "https://beta.pinokio.co/"}, {"title": "Can't install the ffmpeg package · Issue #909 · pinokiocomputer/pinokio", "url": "https://github.com/pinokiocomputer/pinokio/issues/909"}]

Based on the search results, I found information matching your query about Pinokio's features:

Pinokio is an application that helps you install and run complex programs, especially AI tools, on your computer with just one click, eliminating the need to follow long setup guides, open command lines or fix errors.

With one click, the app automatically downloads the app, sets up Python, Conda, virtual environments, GPU libraries and all required dependencies, then launches a local web interface for you in your browser.

Regarding FFMPEG specifically, Pinokio installs the latest version of FFMPEG to support new features, including torchcodec APIs.

This includes built-in support for tools like Git for cloning repositories, Conda and Pip for Python environments, and Node.js with NPM for JavaScript-based projects, all executed through JSON-based scripts that encapsulate complex installation processes such as downloading files, installing libraries, and configuring virtual environments.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

28. assistant (2026-06-21T13:58:16.351Z)

Good. The Pinokio marketing wording IS confirmed ? but it's sourced from the Pinokio website (program.pinokio.computer / the site), not the GitHub README cited. The substance is confirmed; the citation is slightly off. Notably there are open GitHub issues (#908, #909, #910, #1052) where Pinokio's FFMpeg install actually FAILS or installs outdated versions ? a real-world reliability caveat against the "out of the box" framing.

For Ollama: most confirmed, but a few details need nuance ? installer is ~"OllamaSetup.exe" but the ~5MB size and the explicit "auto-detect NVIDIA CUDA" and "offline after" and "models preserved on update" aren't all stated in the docs. Let me verify the Ollama installer size and the ComfyUI desktop claim.

29. user (2026-06-21T13:58:23.929Z)

OllamaSetup.exe Download Size

Based on the releases page provided, **there is no OllamaSetup.exe file listed** in the recent releases shown (v0.30.10 through v0.30.0).

The Windows installer assets that appear are:

- `install.ps1` (22.1 KB)
- `install.sh` (15.5 KB)

These are scripts rather than standalone executables. The largest Windows-related binary assets listed are the `.tar.zst` and `.zip` files, which range from 260 MB to 1.3 GB, but these are not labeled as "OllamaSetup.exe."

To find a Windows installer executable, you may need to check the official Ollama website or a different release page section not shown in this content.

30. user (2026-06-21T13:58:24.277Z)

ComfyUI Desktop App Analysis

****License:**** GPL-3.0 (as indicated in the repository footer)

****Installation formats:**** Yes, the repository confirms it "use[s] the NSIS installer for Windows" and distributes macOS "as a DMG," both as an Electron application.

****Python dependency management:**** Confirmed. The overview states the app bundles "uv" and "will

install all the necessary python dependencies with uv" at startup rather than using embedded Python.

****Pinned requirements:**** Yes, the documentation references ".compiled files" under `assets/requirements/\*.compiled` and describes maintaining "compiled requirements" through specific processes.

****User-selectable models folder:**** Affirmative. Users are "asked to select a location to store ComfyUI files like models, inputs, outputs, custom_nodes and saved workflows."

****Auto-updates:**** The app description confirms it "will automatically update with stable releases of ComfyUI, ComfyUI-Manager (pip), and the uv executable as well as some desktop-specific features."

31. assistant (2026-06-21T13:58:31.246Z)

Claim 4 (ComfyUI desktop) is fully confirmed ? GPL-3.0, NSIS/DMG Electron, bundles uv, .compiled requirements, model folder, auto-updates app+Manager+uv.

For Ollama: the OllamaSetup.exe size of "~5MB" is NOT verified ? the release page shows Windows binaries 260MB?1.3GB. The "~5MB installer" is a common older fact but Ollama installers have grown. The installer downloads/bundles runners. Let me check the actual OllamaSetup.exe size and the auto-detect CUDA / offline claims.

32. user (2026-06-21T13:58:36.943Z)

Web search results for query: "OllamaSetup.exe file size MB download 2025 2026"

Links: [{"title": "Download Ollama for Windows ? Official Installer & Setup Guide 2026", "url": "https://ai-ollama.github.io/"}, {"title": "Download OllamaSetup.exe (Ollama)", "url": "https://sourceforge.net/projects/ollama.mirror/files/v0.13.4/OllamaSetup.exe/download"}, {"title": "Ollama - Browse Files at SourceForge.net", "url": "https://sourceforge.net/projects/ollama.mirror/files/v0.20.7/OllamaSetup.exe/download"}, {"title": "Ollama - Download - Softpedia", "url": "https://www.softpedia.com/get/Artificial-Intelligence-Generative-AI/Ollama.shtml"}, {"title": "Windows - Ollama", "url": "https://docs.ollama.com/windows"}, {"title": "How to Install Ollama on Windows (2026): Step-by-Step | Local AI Master", "url": "https://localaimaster.com/blog/ollama-windows-installation"}, {"title": "Releases · ollama/ollama", "url": "https://github.com/ollama/ollama/releases"}, {"title": "Why is OllamaSetup.exe so large · Issue #8005 · ollama/ollama", "url": "https://github.com/ollama/ollama/issues/8005"}, {"title": "Download Ollama 0.30.8 for Windows | Uptodown.com", "url": "https://ollama.en.uptodown.com/windows/download"}, {"title": "How to Install Ollama on Windows ? Official Step-by-Step Guide 2026", "url": "https://ai-ollama.github.io/install-windows.html"}]

Based on the search results, here's what I found about OllamaSetup.exe file size:

File Size Information

The installer is ~5 MB and has no dependencies. This is confirmed by multiple sources, with the official Windows installer being approximately 5 MB.

However, there's a discrepancy with one source listing a much larger file size. According to Softpedia (updated June 18, 2026), the file size is listed as 1.3 GB, though this appears to be an outlier compared to the official documentation which consistently reports ~5 MB.

Download Details

The installer works on Windows 10 and 11 (64-bit), and Ollama installs in under a minute and starts a background service automatically. It's worth noting that while the installer itself is small, the first pull downloads the model weights (~4-8 GB depending on the model).

The official source for downloading OllamaSetup.exe is from ollama.com.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

33. user (2026-06-21T13:58:37.627Z)

Windows Automatic Installation Requirements

Yes, Python 3.10.6 and git are explicitly listed as prerequisites. Here are the exact installation steps:

****Prerequisites:****

1. "Install Python 3.10.6 (Newer version of Python does not support torch), checking 'Add Python to PATH'."
2. "Install git"

****Installation Steps:****

3. "Download the stable-diffusion-webui repository, for example by running `git clone https://github.com/AUTOMATIC1111/stable-diffusion-webui.git`."
4. "Run `webui-user.bat` from Windows Explorer as normal, non-administrator, user."

The documentation indicates you must manually install Python and git first, then use git clone to download the repository, and finally execute the batch file. The batch script presumably handles virtual environment creation and dependency installation, but the initial Python 3.10.6 and git setup is a separate prerequisite step.

34. assistant (2026-06-21T13:58:44.769Z)

The Ollama ~5MB size has notable nuance: there's an open issue (#8005 "why is OllamaSetup.exe so large") and Softpedia lists 1.3 GB as of June 2026. The "~5MB" was historically true but recent versions appear much larger (the installer now bundles GPU runners). So claim 5's "~5MB" is dated/uncertain. Claim 7 (AUTOMATIC1111) is fully confirmed.

Now let me verify the Fooocus claim (6) and Frigate claims (8, 9).

35. user (2026-06-21T13:58:51.757Z)

Foocus License and Distribution

****License:**** Foocus is licensed under GPL-3.0, as indicated by the "GPL-3.0 license" notation in the repository footer.

****Download Package:**** The Windows distribution is a compressed archive. Users can "directly download Foocus" via a 7z file (version 2.5.0 is ~2.5GB based on the filename format), which they "uncompress" before running.

****Embedded Python:**** The package includes embedded Python, referenced as `.\python\_embeded\python.exe` in the batch file instructions for Windows users.

****First-Run Behavior:**** Upon initial launch, "it will automatically download models" to designated folders. The software then opens a web interface, as Foocus is "based on Gradio" and presents a "localhost web UI."

The exact process: download the archive, extract it, execute `run.bat`, and the application handles model downloading automatically while providing browser-based access to its interface.

36. user (2026-06-21T13:58:53.800Z)

Frigate Distribution and Platform Support

FFmpeg Bundling

The documentation does not specify whether Frigate bundles its own ffmpeg or the version included in the Docker image.

Per-Accelerator Image Variants

Yes, Frigate ships multiple Docker image variants optimized for different hardware:

- `stable` - Standard build for amd64 and RPi-optimized for arm64
- `stable-standard-arm64` - Standard arm64 build
- `stable-tensorrt` - Specialized for amd64 with Nvidia GPUs
- `stable-rocm` - For AMD GPUs
- Community variants like `stable-tensorrt-jp6` (Jetson) and `stable-rk` (Rockchip)

Distribution Channels

Frigate is distributed via Docker tags on GHCR and as a Home Assistant add-on with multiple variants available.

Windows Support

Windows is not officially supported. The documentation states: `"Windows is not officially supported, but some users have had success getting it to run under WSL or Virtualbox."` However, acquiring proper GPU and Coral device access `"may be difficult or impossible."`

37. assistant (2026-06-21T13:58:59.147Z)

Claim 6 (Fooocus): confirmed ? GPL-3.0, embedded Python ZIP/7z (~2.5GB), run.bat, downloads models first run, Gradio localhost UI. The "~1GB+" is accurate (it's actually ~2.5GB).

Claim 9 (Frigate Windows): confirmed ? not officially supported, WSL/Virtualbox, GPU/Coral "difficult or impossible."

Claim 8 (Frigate): partial ? per-accelerator variants confirmed, GHCR + HA add-on confirmed. But the doc this fetch hit didn't confirm the ffmpeg bundling (at /usr/lib/ffmpeg, v7) or the model_cache detail. Also the claim says "default OpenVINO/CPU" but the variant list shows `stable` as the default. Let me verify the ffmpeg bundling and detector model_cache facts, plus the LM Studio / Jan / GPT4All claim (10).

38. user (2026-06-21T13:59:05.225Z)

Jan: Architecture and Features

****Tauri-based desktop app:**** Yes. The acknowledgements section credits "Tauri" as one of the giants the project is built upon, and the prerequisites mention "Rust (for Tauri)."

****llama.cpp bundling:**** Yes. The project acknowledges "Llama.cpp" in its foundations, listing it among the core technologies it relies on.

****Local OpenAI-compatible server:**** Yes, confirmed. The features describe an "OpenAI-Compatible API: Local server at `localhost:1337`" for use by other applications.

All three components are confirmed in the repository documentation, with the local server explicitly specified to run on port 1337.

39. user (2026-06-21T13:59:07.509Z)

Web search results for query: "Frigate docker bundled ffmpeg /usr/lib/ffmpeg version 7 model_cache ONNX first boot detectors"

Links: [{"title": "Frigate + Reolink + NVIDIA Setup Guide (2026): Docker, go2rtc & TensorRT", "url": "https://corelab.tech/setupfrigate/"}, {"title": "[Detector Support]: frigate 0.17 building onnx model · blakeblackshear/frigate · Discussion #22201", "url": "https://github.com/blakeblackshear/frigate/discussions/22201"}, {"title": "Object Detectors | Frigate", "url": "https://docs.frigate.video/configuration/object_detectors/"}, {"title": "Frigate NVR - Complete Setup Guide with Nvidia Blackwell | Flaviu Vlaicu ?", "url": "https://vlaicu.io/posts/frigate-guide/"}, {"title": "Docker Configuration and Multi-Platform Builds | blakeblackshear/frigate | DeepWiki", "url": "https://deepwiki.com/blakeblackshear/frigate/10.1-installation-methods-and-"}]

```
requirements"}, {"title": "[HW Accel Support]: FFMPEG not running on second GPU ·  
blakeblackshear/frigate · Discussion #21467", "url": "https://github.com/blakeblackshear/frigate/d  
iscussions/21467"}, {"title": "[Support]: ffmpeg missing from Docker container ·  
blakeblackshear/frigate · Discussion  
#16892", "url": "https://github.com/blakeblackshear/frigate/discussions/16892"}, {"title": "Releases  
· blakeblackshear/frigate", "url": "https://github.com/blakeblackshear/frigate/releases"}, {"title":  
:"Frigate 0.16 Installation Tutorial - The Bearded  
Tek", "url": "https://beardedtek.org/frigate-0-16-installation-tutorial/"}]
```

Based on the search results, I found relevant information about Frigate's Docker FFmpeg bundling, ONNX detectors, and the first-boot model caching process:

Key Findings:

****Bundled FFmpeg Versions:****

Frigate provides bundled FFmpeg 5.0 and 7.0 binaries. The container provides binaries in /usr/lib/ffmpeg, which aligns with the path in your query.

****ONNX Detectors and First Boot:****

ONNX is an open format for building machine learning models, Frigate supports running ONNX models on CPU, OpenVINO, ROCm, and TensorRT. On startup Frigate will automatically try to use a GPU if one is available.

For AMD GPU users specifically: The recommended approach is to disable object detection in the config, then startup Frigate with the onnx detector configured, where the main object detection model will be converted to mxr format and cached in the config directory.

****First Boot Model Caching:****

First boot with a new model takes extra time while TensorRT builds an optimized engine for your specific GPU. This is a one-time process per model.

The search results confirm that the Frigate Docker image uses /usr/lib/ffmpeg/7.0/bin/ffmpeg path for FFmpeg 7.0, and that ONNX models are cached in the /config/model_cache/ directory during the first boot initialization process.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

40. assistant (2026-06-21T13:59:12.520Z)

Claim 8 (Frigate ffmpeg bundling): confirmed ? /usr/lib/ffmpeg with v7 (specifically /usr/lib/ffmpeg/7.0/bin/ffmpeg), model_cache in /config, ONNX converted on first boot. The "default OpenVINO/CPU" framing is slightly loose (the default `stable` tag includes multiple CPU detectors incl. OpenVINO), but the substance holds.

Claim 10 (Jan): Tauri + llama.cpp + localhost:1337 all confirmed. Now let me verify the LM Studio (llmster daemon, LM Runtimes) and GPT4All (:4891 server, HF model hub) parts of claim 10.

41. user (2026-06-21T13:59:17.961Z)

GPT4All: Installer, Model Hub, and API Server

****OS Installer:**** Yes, GPT4All provides installers for multiple operating systems. The page states: "Install GPT4All for your operating system and open the application" with downloads available for Windows, Mac, and Linux.

****In-App Model Hub:**** Yes, GPT4All includes a built-in model browser. Users can "Click `+ Add Model`" and "Download a model." The documentation mentions you can "browse models available online to download onto your device."

****Local API Server:**** The page confirms an API server exists. There is a dedicated section titled "API Server" with a link to "GPT4All API Server" documentation. However, ****the specific**

port number is not mentioned** in this content excerpt.

42. user (2026-06-21T13:59:25.045Z)

Web search results for query: "LM Studio 0.4 headless daemon llmster OpenAI-compatible server LM Runtimes swappable inference engine"

Links: [{"title": "Run LM Studio as a service (headless) | LM Studio", "url": "https://lmstudio.ai/docs/developer/core/headless"}, {"title": "Setup llmster as a Startup Task on Linux | LM Studio", "url": "https://lmstudio.ai/docs/developer/core/headless_llmster"}, {"title": "How to Run LLMs Locally with LM Studio: Complete Guide 2026 | by Kapustin Mykhailo | Medium", "url": "https://medium.com/@kapustinomm/how-to-run-llms-locally-with-lm-studio-complete-guide-2026-c941843e000e"}, {"title": "LM Studio Developer Docs | LM Studio", "url": "https://lmstudio.ai/docs/developer"}, {"title": "LM Studio Tips & Tricks: Hidden Features | InsiderLLM", "url": "https://insiderllm.com/guides/lm-studio-tips-and-tricks/"}, {"title": "docs/1_developer/0_core/headless.md at main · lmstudio-ai/docs", "url": "https://github.com/lmstudio-ai/docs/blob/main/1_developer/0_core/headless.md"}, {"title": "LM Studio Complete Guide (2026): Run Local LLMs With a Real GUI", "url": "https://codersera.com/blog/lm-studio-complete-guide-2026/"}, {"title": "LM Studio, llmster, and lms | LM Studio", "url": "https://lmstudio.ai/docs/app/basics/lmstudio-vs-llmster-vs-lms"}, {"title": "LM Studio · OpenClaw", "url": "https://docs.openclaw.ai/providers/lmstudio"}, {"title": "LM Studio 0.4 Headless Deployment: Local LLM APIs Without the GUI", "url": "https://www.sitepoint.com/lm-studio-04-headless-deployment-local-llm-apis-without-the-gui/"}]

Based on the search results, here's what I found about LM Studio 0.4 and its headless components:

Key Components

llmster is the recommended standalone daemon for LM Studio, requiring no GUI. llmster is LM Studio's core, packaged as a daemon for headless deployment on servers, cloud instances, or CI.

LM Studio 0.4 Features

Version 0.4 added parallel requests, a standalone headless daemon (llmster), and remote access via LM Link. LM Studio 0.4 headless deployment solves the core friction point that until now, LM Studio required a desktop GUI, making it unsuitable for remote servers and automated workflows. The 0.4 release introduces a fully headless mode driven by the lms CLI, enabling developers to download models, configure inference parameters, and launch OpenAI-compatible API servers entirely from the command line.

OpenAI-Compatible Server

A REST API on `http://localhost:1234` mirrors OpenAI's `/v1/chat/completions`, `/v1/completions`, `/v1/embeddings` and `/v1/models` endpoints. Any OpenAI SDK or tool works against it by swapping the base URL.

Swappable Inference Engines

The underlying engines (llama.cpp, MLX) are open source. LM Studio allows you to choose between these different backends for inference, with Apple Silicon using Apple's MLX backend, which is typically 30 to 50 percent faster than llama.cpp on Metal.

The search results confirm all the key components mentioned in your query are real features of LM Studio 0.4.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

43. assistant (2026-06-21T13:59:31.691Z)

Claim 10 verified with one nuance: LM Studio's llmster daemon and "LM Runtimes" (swappable engines llama.cpp/MLX) are confirmed; GPT4All installer + in-app model hub confirmed. The specific port numbers ? LM Studio's OpenAI server is documented as :1234 (the claim doesn't pin LM Studio's port, so fine), and GPT4All's :4891 wasn't confirmed on the quickstart page (it's the documented default elsewhere, so plausible but the page didn't state it). These are minor and don't undercut the claim's thrust ("all bundle the engine and download weights on demand via a GUI").

Let me do one final check: confirm the production stitcher actually uses libx264 (to nail the recommendation's "no GPL x264 per D4" being contradicted by current code), and verify D10's exact location/wording.

44. user (2026-06-21T13:59:33.122Z)

```
245         trim_duration = padded_end - padded_start
246         logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
247
248         base_cmd = [
249             "ffmpeg", "-y",
250             "-ss", str(round(padded_start, 3)),
251             "-to", str(round(padded_end, 3)),
252             "-i", raw_video_path,
253             "-c:v", "libx264",
254             "-preset", "superfast",
255             "-crf", "22",
256             "-c:a", "aac",
257             "-b:a", "128k",
258         ]
259         vf_args = ["-vf", watermark_filter] if watermark_filter else []
260
261         # Try with the watermark first; if that encode fails (e.g. a bad font/logo
262         # path), retry once WITHOUT it so a branding misconfig can never nuke the
263         # whole reel. Clips stay codec-identical either way, so concat -c copy holds.
264         attempts = [vf_args, []] if vf_args else [[]]
```

45. user (2026-06-21T13:59:34.160Z)

39 | D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the tool | Owner 2026-06-20 | **supersedes `DESKTOP\_APP\_PLAN.md`**; one artifact (this appliance) runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |

46. assistant (2026-06-21T13:59:39.303Z)

All confirmed. D10 is exactly as claimed at line 39 (claim cited line 39 ? correct). The production stitcher uses libx264 (GPL encoder) hardcoded at compiler.py:253, confirming the recommendation's "no GPL x264 per D4" both mis-cites D4 and conflicts with current shipped code.

I have everything needed. Let me compile my structured verification.

47. user (2026-06-21T14:01:05.242Z)

Structured output provided successfully

48. assistant (2026-06-21T14:01:11.463Z)

Verification complete. I adversarially checked all 12 claims against engine source, the khelsutra docs, and current web facts. Summary: 9 confirmed, 2 uncertain (native-WASB "not production-verified" understates a recorded RTX-2070 parity run; Ollama "~5MB" is stale ? current installers are 260MB?1.3GB), and 1 refuted (a Cloud Run Dockerfile now exists at `deploy/cloudrun/Dockerfile`, shipped ~1 hour after the khelsutra doc claiming "no Dockerfile" was written).

The two load-bearing problems with the recommendation itself: (1) finishing ``native_wasb_runner.py`` does NOT collapse to one torch env ? it still shells out to a separate ``wasb`` conda env (py3.8/torch 1.11), as the new M6 Dockerfile makes explicit by deliberately keeping two python stacks; and (2) the cited "no GPL x264 per D4" mis-references D10's own doc (where D4 is "embed RallyPicker"), actually belongs to the superseded ``DESKTOP_APP_PLAN.md``, and conflicts with current code that hardcodes `libx264`.